

READINGQUEST



Okeke Onyedi Joachim

Software Architect – 2025 (2024 – 2026)

Indice

Sommario.....	3
Prospettiva del prodotto.....	3
Funzioni principali del sistema.....	4
Vincoli del sistema.....	4
Ipotesi e dipendenze.....	5
Requisiti funzionali.....	5
Requisiti non funzionali.....	7
Requisiti di interfaccia.....	7
Vincoli tecnici.....	8
Use cases.....	9
Architettura tecnica del sistema.....	13
Relazione tra i componenti.....	14
Entità principali del sistema.....	16
Wireframe.....	19
Estensioni future.....	25

Sommario

L'applicazione "ReadingQuest" nasce come incentivo e stimolante per la quotidianità di lettori o aspiranti-lettori i quali trovano difficoltoso mantenere quest'abitudine o che desidererebbero rendere questo hobby più intrattenente e stimolante, attraverso un'estensione dei loro progressi su dispositivi mobili Android.

Molte volte siamo intimoriti ad iniziare un nuovo libro, inconsapevoli del tempo che verrebbe richiesto per completarlo. Se si avessero un numero totale di pagine da leggere quotidianamente, questa paura scemerebbe. Sarebbe possibile fare ciò tranquillamente su carta, ma essendo diventato i dispositivi mobile un'estensione della persona, ed essendo una semplice nota su carta una task a basso responso dopaminico, ritengo che un'applicazione che fa maturare un avatar fittizio con monete ed XP - seguendo il percorso RPG, sia la migliore strada per rendere molto più intrattenente, un passatempo che sta lentamente diminuendo.

Prospettiva del prodotto

ReadingQuest è un'applicazione mobile nativa per dispositivi Android, sviluppata in linguaggio Java e basata su un database locale SQLite.

L'applicazione opera completamente offline. Tutti i dati relativi ai libri, ai progressi e ai risultati dei test sono memorizzati localmente.

L'app si posiziona nel dominio delle applicazioni educative e di autosviluppo, ma integra elementi tipici del game design – missioni, progressione, XP, avatar, ricompense – con l'obiettivo di rendere la lettura un processo più immediato e appagante.

Funzioni principali del sistema

Le 3 macro-funzionalità fondamentali:

1. Gestione utente

- Registrazione e login tramite credenziali locali
- Salvataggio persistente del profilo (XP, monete, velocità di lettura)

2. Misurazione della velocità di lettura

- Test suddiviso in 5 pagine tematiche
- Timer integrato
- Calcolo automatico della velocità media
- Salvataggio dei risultati per future stime

3. Gestione dei libri

- Importazione manuale di libri
- Suddivisione in quattro categorie: “Tutti”, “In lettura”, “Letti”, “Da leggere”
- Calcolo del tempo necessario per completare ogni libro
- Possibilità di impostare missioni con XP e monete (sarà automatico, nella versione non beta)

Vincoli del sistema

- Il sistema funziona interamente offline
- L'applicazione è Java native
- Il database scelto è SQLite, integrato direttamente nel dispositivo

- La navigazione avviene tramite Activities dichiarate nel Manifest
 - L'interfaccia deve risultare intuitiva e facilmente navigabile da utenti non esperti
-

Ipotesi e Dipendenze

Per il corretto funzionamento del sistema si assumono le seguenti condizioni:

- L'utente inserisce dati veritieri durante il test di lettura
 - Il dispositivo Android consente la scrittura nella memoria locale
 - L'utente aggiorna regolarmente lo stato dei libri
 - L'app non richiede connettività per le sue funzionalità principali
-

Requisiti Funzionali (RF)

Registrazione dell'utente

Il sistema deve permettere all'utente di creare un nuovo profilo, inserendo i dati richiesti (nome, cognome, username, password).

L'inserimento deve essere validato e salvato nella tabella *users* del database SQLite.

Autenticazione tramite credenziali

L'utente deve poter accedere tramite username e password.

Il sistema deve verificare la corrispondenza delle credenziali con quelle salvate in locale.

Esecuzione del test di velocità di lettura

L'utente deve poter eseguire un test composto da cinque pagine tematiche. Ogni pagina deve poter essere consultata singolarmente, con un timer che misura il tempo totale di lettura.

Visualizzazione dei libri

Il sistema deve mostrare la libreria dell'utente suddivisa in quattro categorie:

- Tutti i libri
- In lettura
- Da leggere
- Letti

Aggiunta di un nuovo libro

L'utente deve poter aggiungere manualmente un libro inserendo titolo, autore, genere, pagine o ISBN.

Calcolo del tempo stimato per la lettura di un libro

Per ogni libro inserito, il sistema deve mostrare una stima del tempo necessario per completarlo, basata sulla velocità misurata dell'utente.

Impostazione di una missione di lettura

L'utente può impostare un libro come "missione", associando una deadline che indichi il numero di pagine da leggere quotidianamente per riuscire a stare nel tempo prestabilito.

Alla conclusione della missione, il sistema assegna XP e monete, aggiornando la tabella *book_progress*.

Persistenza dei dati dell'utente e dei progressi

Tutte le informazioni (test, libri, missioni, XP, monete) devono essere salvate localmente e recuperate automaticamente all'avvio dell'app.

Requisiti Non Funzionali (RNF)

Usabilità

L'interfaccia deve risultare semplice e intuitiva, con una navigazione lineare e pulsanti chiaramente identificabili.

L'utente deve poter completare le principali azioni (aggiungere un libro, fare il test, visualizzare la libreria) in pochi passaggi.

Prestazioni e reattività dell'app

L'app deve rispondere entro pochi millisecondi per operazioni locali come:

- caricamento schermate
- accesso al database
- aggiornamento degli stati

Nota: data la natura offline e locale di questa versione beta, non dovrebbe essere percepito alcun ritardo significativo.

Affidabilità della persistenza dei dati

Il database SQLite deve garantire la memorizzazione permanente dei dati anche in caso di chiusura improvvisa dell'app.

Requisiti di Interfaccia (RI)

Coerenza grafica

Tutte le schermate devono mantenere uno stile uniforme: colori, font, pulsanti e margini devono essere coerenti.

Navigazione semplice tra le schermate

Ogni Activity deve permettere di tornare alla schermata precedente tramite pulsante o back gesture.

Liste presentate con RecyclerView

Le liste dei libri devono essere gestite tramite RecyclerView per migliorare prestazioni e scorrimento fluido.

Feedback all'utente

Ogni azione importante (aggiunta libro, conclusione missione, aggiornamento stato) deve essere accompagnata da un feedback visivo, come Toast o cambi di colore.

Vincoli Tecnici

Linguaggio e piattaforma

L'applicazione deve essere sviluppata in **Java**, utilizzando **Android Studio**.

Database locale

Il sistema utilizza esclusivamente **SQLite** come tecnologia di persistenza.

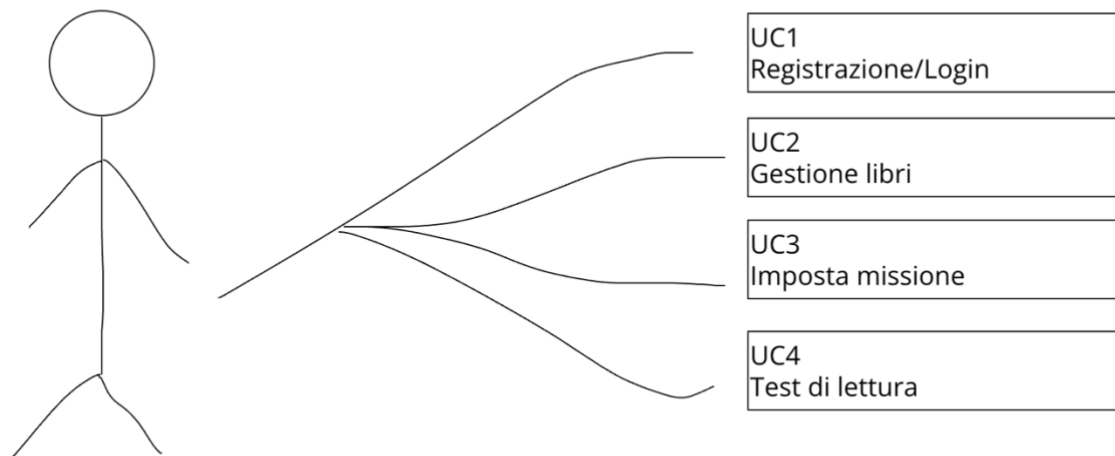
Architettura interna

Il progetto segue una suddivisione in:

- Activities per la logica di navigazione

- Repository per la gestione dei dati
 - DatabaseHelper per la creazione e manipolazione del database
 - Modelli Java per rappresentare le entità (User, Book, BookProgress, ReadingTest)
-

Use cases



- Tutti i flussi partono dall'utente
- Dopo UC1, l'utente accede alle funzionalità principali
- La gestione missioni dipende dalla gestione libri
- Il test di lettura è un caso d'uso autonomo, ma influenza le stime di lettura

UC1 – Registrazione e Autenticazione

Attore principale: Utente

Precondizioni:

- L'applicazione è installata sul dispositivo
- L'utente non è autenticato

Scenario principale:

1. L'utente avvia l'app e seleziona "Registrati".
2. Il sistema mostra il modulo per inserire nome, cognome, username e password.
3. L'utente inserisce i dati richiesti.
4. Il sistema valida i campi e salva il nuovo utente in SQLite.
5. L'utente torna alla schermata di login.
6. L'utente inserisce username e password.
7. Il sistema verifica le credenziali.
8. L'utente accede alla *Landing Page* dell'app.

Postcondizioni:

- L'utente è autenticato.
- Le sue informazioni sono memorizzate localmente.

UC2 – Gestione Libreria

Attore principale: Utente

Precondizioni:

- L'utente è autenticato
- Sono presenti uno o più libri nel database, oppure l'utente può aggiungerne

Scenario principale:

1. L'utente apre la sezione "Libri".
2. Il sistema mostra quattro categorie:
 - Tutti
 - In lettura
 - Da leggere
 - Letti
3. L'utente seleziona una categoria.
4. Il sistema interroga la tabella *books* e mostra la lista corrispondente tramite RecyclerView.
5. L'utente seleziona un libro.
6. Il sistema mostra i dettagli: autore, pagine, stato, tempo stimato di completamento.

Estensioni (scenari alternativi):

- **Aggiunta di un libro:**

1. L'utente preme "Aggiungi Libro".
2. Inserisce titolo, autore, genere, ISBN/pagine.
3. Il sistema salva il libro nella tabella *books*.

- **Cambio stato libro:**

L'utente può impostare il libro come "In lettura", "Letto" o "Da leggere".

Postcondizioni:

- Lo stato della libreria è aggiornato.

UC3 – Imposta Missione

Attore principale: Utente

Precondizioni:

- Deve esistere almeno un libro in stato “In lettura”.
- L’utente è autenticato.

Scenario principale:

1. L’utente seleziona un libro dalla lista “In lettura”.
2. Il sistema apre la schermata della missione.
3. L’utente seleziona una deadline.
4. L’utente conferma la missione.
5. Il sistema crea un record nella tabella *book_progress* con stato “missione attiva”.

Scenario successivo (conclusione missione):

1. L’utente marca il libro come “Letto”.
2. Il sistema assegna XP e monete in base alle regole interne.
3. La missione viene segnata come completata.

Postcondizioni:

- XP e monete dell’utente sono aggiornati.
 - Lo stato del libro è aggiornato a “Letto”.
-

Architettura tecnica del Sistema

Introduzione all'Architettura

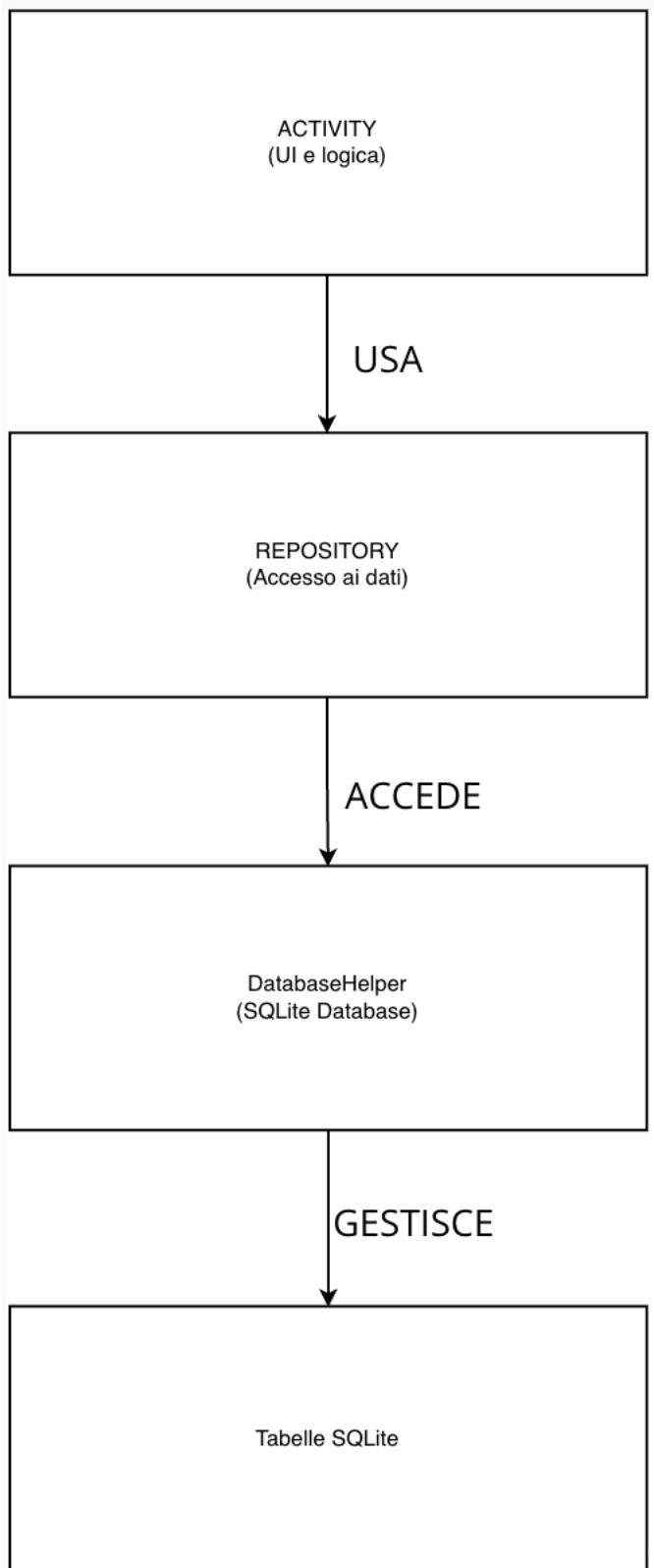
L'applicazione segue un approccio client-side offline, in cui tutti i dati (libri, utenti, test, missioni) risiedono localmente sul dispositivo dell'utente attraverso un database SQLite.

3 livelli funzionali:

1. **Presentazione** → Activities + XML layout
2. **Logica di accesso ai dati** → Repository
3. **Persistenza e modello dati** → SQLite + DatabaseHelper + classi di modello

Questa separazione dei ruoli garantisce: (1) codice mantenibile, (2) componenti riutilizzabili, (3) più facile debug, (4) riduzione degli accoppiamenti tra interfaccia e accesso ai dati

Relazione tra i componenti



L'utente interagisce con una Activity

e.g. BooksActivity.

L'Activity richiede i dati al Repository

e.g.

```
bookRepository.getAllBooks();
```

Il Repository interroga SQLite tramite DatabaseHelper

- apre connessione
- esegue query
- ottiene Cursor
- converte righe in oggetti Book

Il Repository restituisce una lista di oggetti Java

e.g. `List<Book> books`

L'Activity aggiorna l'interfaccia

p.e. con un RecyclerView Adapter.

Motivazione delle scelte architettureali

- Necessità di funzionamento offline → L'utilizzo di SQLite garantisce rapidità, persistenza locale e assenza di dipendenze esterne.
- Necessità di separare la logica dall'interfaccia → L'introduzione del Repository permette di evitare SQL all'interno delle Activities, migliorando leggibilità e manutenibilità.
- Necessità di avere un progetto scalabile → In futuro si potranno aggiungere schermate, nuovi moduli (p.e. leaderboard online) e funzionalità senza riscrivere tutto il sistema.

Entità principali del sistema

Le entità presenti nel database rappresentano i concetti fondamentali dell'applicazione:

- **User** → il profilo dell'utente
- **Book** → i libri che l'utente gestisce
- **BookProgress** → le missioni e lo stato dei libri
- **ReadingTest** → i risultati del test di lettura

Entità: User

Rappresenta il profilo personale dell'utente.

Attributi principali:

- id : long
- nome : String
- cognome : String
- dataNascita : String
- username : String
- password : String
- avgReadingSpeed : double
- alterEgoTitle : String
- monete : int
- xp : int
- badge : String

Ruolo nel sistema: principale da cui dipendono i progressi, le missioni e i risultati dei test.

Entità: Book

Rappresenta un libro inserito dall'utente.

Attributi principali:

- id : long
- userId : long
- titolo : String
- autore : String
- genere : String
- casaEditrice : String
- isbn : String
- pagineTotali : int

Ruolo nel sistema:

È l'oggetto su cui si basano: (1)le missioni, (2)i progressi di lettura, (3)i calcoli di tempo stimato

Entità: BookProgress

Gestisce la progressione di lettura dei libri.

Attributi principali:

- id : long
- bookId : long (nome esatto, NON fk_book)
- status : String
- startDate : String
- endDate : String
- missione : String (stringa, non booleano)
- xpReward : int
- moneteReward : int
- pagesRead : int

Ruolo nel sistema:

Rappresenta lo *storico* e lo *stato attuale* della missione.

Permette di sapere: (1)se un libro è parte di una missione, (2)quando è stato iniziato, (3)se è stato completato, (4)quanti XP e monete assegnare

Entità: ReadingTest

Rappresenta i risultati dei test di lettura effettuati dall'utente.

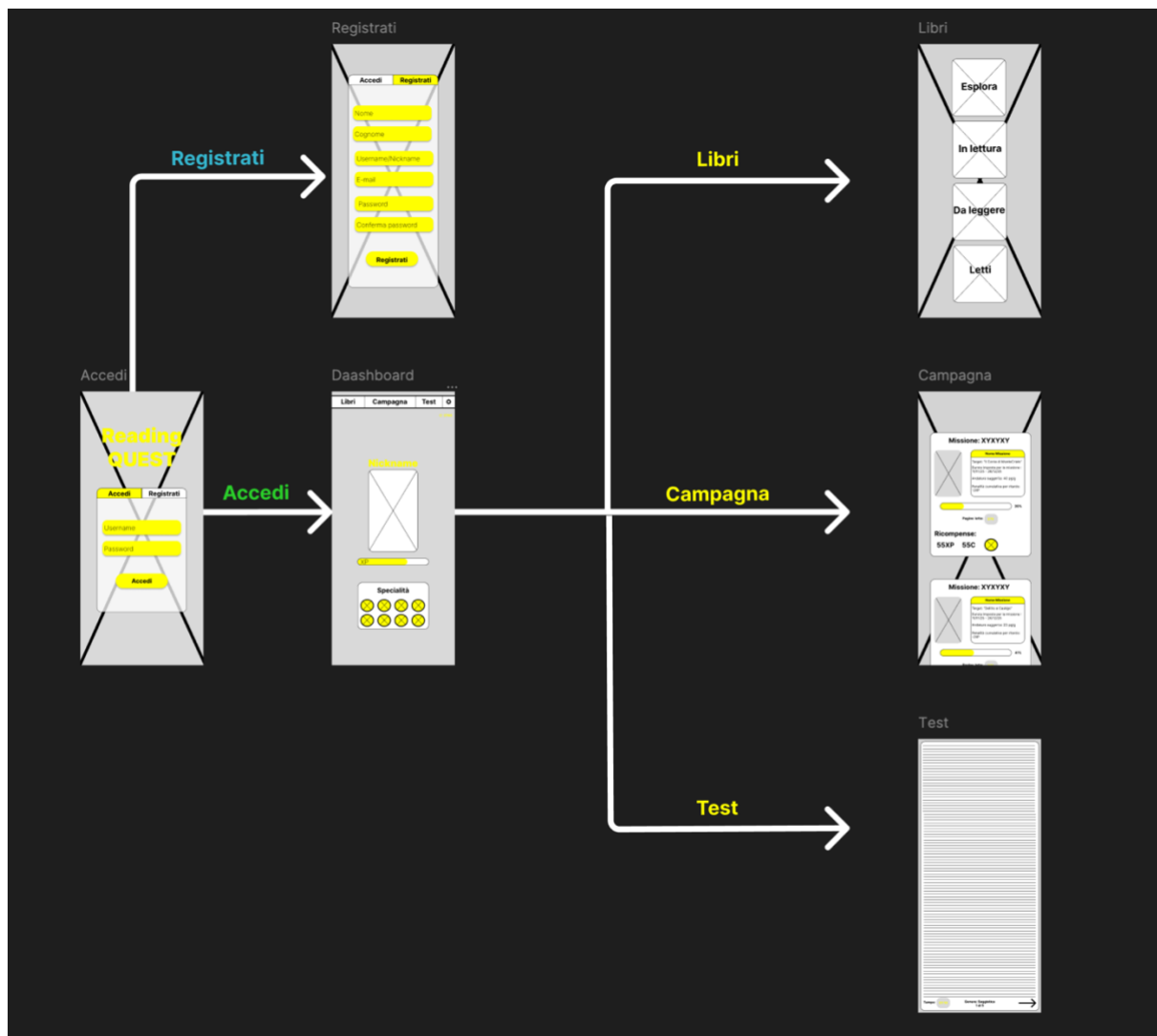
Attributi principali:

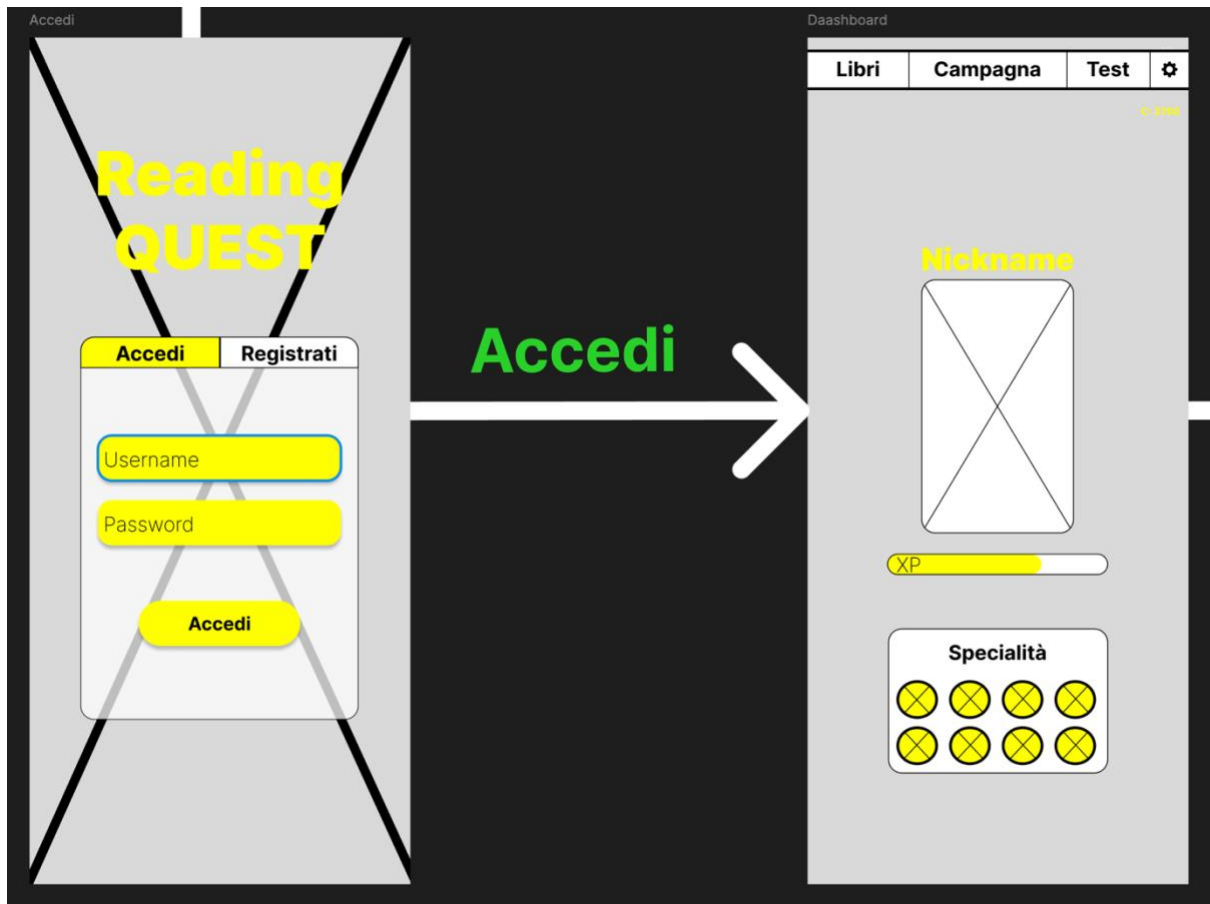
- id : long
- userId : long
- tempoTotaleMs : long
- mediaVelocita : double
- creatoll : String

Ruolo nel sistema:

I risultati di questa tabella determinano la **velocità media** dell'utente, che serve per calcolare la stima di lettura dei libri.

Wireframe





Registrati



Registrati

Accedi

Registrati

Nome

Cognome

Username/Nickname

E-mail

Password

Conferma password

Registrati

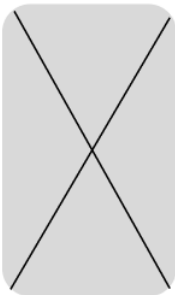
Esplora

In lettura

Da leggere

Letti

Missione: XYXYXY



Nome Missione

Target: "Il Conte di MonteCristo"
Durata imposta per la missione :
11/11/25 - 26/12/25
[Andatura suggerita: 40 pg/g](#)
Penalità cumulativa per ritardo:
-2XP



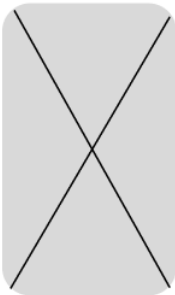
Pagine lette: 378

Ricompense:

55XP 55C



Missione: XYXYXY



Nome Missione

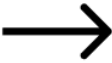
Target: "Delitto e Castigo"
Durata imposta per la missione :
11/11/25 - 26/12/25
Andatura suggerita: 23 pg/g
Penalità cumulativa per ritardo:
-2XP



Pagine lette: 255

Tempo: 01:16

Genere: Saggistica
1 di 5



Estensioni future

Classifiche online (Leaderboard)

Implementare una classifica globale che permetta ai lettori di confrontare:

- XP guadagnati nella season di battaglia in solo
- XP guadagnati nella season di battaglia in party

Richiederebbe un backend remoto e sincronizzazione online (Phoenix).

Party di Lettura

Creare gruppi sociali con un massimo di 5 persone, dove si possono:

- svolgere raid ai dungeon per battere boss che richiedano un misto di libri da leggere per poterlo sconfiggere

Avatar personalizzabile

Espandere il sistema di gamification introducendo:

- classi dipendentemente dal genere di libri letti
- customizzazione del personaggio (three.js)